System Verification and Validation Plan for Squigglr

Team #17, Squigglr
Rayan Nasrallah
Ahmed Elzaria
Muhammad Huzaifah
Shamil Canbolat
Moustafa Moustafa

November 1, 2026

Revision History

Date	Developer(s)	Change
29 October 2026	Rayan Nasrallah	Wrote Section 3.6, Automated Testing and Verification Tools (#222).
29 October 2026	Rayan Nasrallah	Wrote the performance tests in Section 4.2, T-PERF-001 to T-PERF-007 (#223).
29 October 2026	Rayan Nasrallah	Wrote the safety-critical tests in Section 4.2, T-SAF-001 to T-SAF-017, and the safety inspection (#226).
29 October 2026	Rayan Nasrallah	Wrote the operational, maintainability, security, cultural and compliance tests in Section 4.2, the traceability table in Section 4.3 and the Symbolic Parameters in Section 6.1 (#227).
29 October 2026	Rayan Nasrallah	Added his reflection answers (#228).
29 October 2026	Rayan Nasrallah	Added his generative AI use disclosure (#229).
30 October 2026	Moustafa Moustafa	Wrote Section 3.1, Verification and Validation Team (#232).
30 October 2026	Moustafa Moustafa	Wrote Sections 2.1 and 2.2, Summary and Objectives (#233).
30 October 2026	Moustafa Moustafa	Wrote Section 2.4, Relevant Documentation, and added bibliography entries for the project documents (#234).
30 October 2026	Moustafa Moustafa	Wrote Section 3.4, Verification and Validation Plan Verification (#235).
30 October 2026	Moustafa Moustafa	Wrote the revision history, Section 1, Symbols, Abbreviations, and Acronyms, and the document roadmap; added Rayan's revision history rows to the SRS and Hazard Analysis (#236).
30 October 2026	Moustafa Moustafa	Wrote the introductions to Sections 3, 4, 4.1, 4.2 and 4.3 (#237).

Date	Developer(s)	Change
30 October 2026	Moustafa Moustafa	Rewrote reflection Questions 1 and 2 as team answers and added Moustafa's Questions 3 and 4 (#242).

Contents

1 Symbols, Abbreviations, and Acronyms **v**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 2
- 2.3 Custom Documents . . . 3
- 2.4 Relevant Documentation . . . 4

3 Plan **5**
- 3.1 Verification and Validation Team . . . 5
- 3.2 SRS Verification . . . 7
- 3.3 Design Verification . . . 8
- 3.4 Verification and Validation Plan Verification . . . 8
- 3.5 Implementation Verification . . . 11
- 3.6 Automated Testing and Verification Tools . . . 19
- 3.7 Software Validation . . . 21

4 System Tests **22**
- 4.1 Tests for Functional Requirements . . . 22
  - 4.1.1 Area of Testing1 . . . 23
  - 4.1.2 Area of Testing2 . . . 24
- 4.2 Tests for Nonfunctional Requirements . . . 24
  - 4.2.1 Performance . . . 24
  - 4.2.2 Safety-Critical Requirements . . . 30
  - 4.2.3 Operational and Environmental . . . 43
  - 4.2.4 Maintainability and Support . . . 44
  - 4.2.5 Security . . . 47
  - 4.2.6 Cultural . . . 50
  - 4.2.7 Compliance . . . 51
- 4.3 Traceability Between Test Cases and Requirements . . . 52

5 Unit Test Description **54**
- 5.1 Unit Testing Scope . . . 54
- 5.2 Tests for Functional Requirements . . . 54
  - 5.2.1 Module 1 . . . 55
  - 5.2.2 Module 2 . . . 56

5.3 Tests for Nonfunctional Requirements . . . 56
5.3.1 Module ? . . . 56
5.3.2 Module ? . . . 56
5.4 Traceability Between Test Cases and Modules . . . 57

6 Appendix 58
6.1 Symbolic Parameters . . . 58
6.2 Usability Survey Questions? . . . 59

7 Appendix — Generative AI Use Disclosure 60
7.1 AI Tools Used . . . 60
7.2 How and Where Used . . . 60
7.3 Verification of AI-Generated Content . . . 60

8 Appendix — Reflection 62

List of Tables

1 Abbreviations and terms added by this plan . . . v
2 Verification and validation roles of the team members . . . 6
3 Reviews of this plan . . . 8
4 Static verification of the implementation . . . 12
5 Code Checklist items and how reviewers apply them . . . 14
6 Automated testing and verification tools . . . 20
7 Traceability for nonfunctional requirements . . . 53
8 Symbolic parameters for nonfunctional tests . . . 58

1 Symbols, Abbreviations, and Acronyms

We use the terms and acronyms in the glossary of the Software Requirements Specification (SRS Section 4.1) and the symbolic constants in SRS Section 4.2 without defining them again here. Table 1 lists only the ones this plan adds. Constants that only our tests use are in the Symbolic Parameters appendix.

Table 1: Abbreviations and terms added by this plan

Term	Meaning
DevOps	Development and Operations: development environments, build automation and deployment.
Parameterized test	One test with a table of cases, where each case has a label and its own expected output, so it can be cited on its own, for example `T-LRN-005 (c)`.
PH-1, PH-2, PH-3	The SRS delivery phases: the proof-of-concept demo (23 November 2026), Rev 0 (1 February 2027) and the final demo (15 March 2027).
PR	Pull request: a proposed change to the repository, reviewed before it merges into `main`.
Rev 0, Rev 1	The first complete version of the product and documents, and the final revised version.
SAF	Prefix of the SRS safety-critical requirements, for example `SAF-RAY-001`.
SI	Safety inspection: the code inspection that gives each safety requirement a second, independent check (Section 4.2.2).
`T-<AREA>-###`	A system test ID. `AREA` is the requirement group it tests: `LRN`, `REC`, `CRT`, `PRG`, `UX`, `PERF`, `SAF`, `OPS`, `MAINT`, `SEC`, `CULT` or `COMP`.
VnV	Verification and validation.

This document is our plan for verifying and validating Squigglr: how we will check that it is built correctly, and that it is the right product for children and the adults supporting them. Section 2 says what we are testing and what we are focusing on. Section 3 says who does what, how we will review the SRS, the design, this plan and the code, which tools we use, and how we will validate Squigglr with the supervisor, the specialists and adult users. Section 4 gives the system tests for the functional and nonfunctional requirements and traces each test to its requirement. Section 5 explains how we will choose unit tests; the module tests are added once the Module Interface Specification (MIS) is done. The appendices hold the symbolic parameters, the usability survey questions, our generative AI use disclosure and our reflection.

2 General Information

2.1 Summary

Squigglr is a website that helps children practise drawing on a touchscreen with a stylus, and helps the adults supporting them see how that practice is going. This plan covers its four main parts:

- **Guided lessons.** An adult picks a lesson, like drawing a square, and the child works through stages that give less and less help: a demonstration, tracing a full outline, drawing between endpoints, then drawing with no guide. Squigglr moves to the next stage when an attempt passes. Squigglr can suggest going back a stage, but only an adult can make that change.
- **Creative drawing.** The child draws freely, and Squigglr can tidy up the drawing by adjusting the child's own strokes. The result stays editable and the original can always be restored.
- **Attempt records.** Each attempt keeps the stage, the difficulties the child ran into and the corrections they made, not just the final drawing.
- **Progress summaries.** Adults see how a child is doing over time, with attempts made with and without a guide kept separate.

The full requirements are in the SRS.

2.2 Objectives

Our main objective is to build confidence that Squigglr is safe for children, is actually what they and the adults supporting them need, and behaves correctly. In order of priority:

1. **Keep children safe and protect their well-being.** Squigglr is used by young children, some with autism, attention-deficit/hyperactivity disorder (ADHD) or other learning needs, so a failure that frustrates or upsets a child matters more than most bugs. Every safety-critical requirement (SRS Section 12.2) gets a test and a second, independent check in the safety inspection (Section 4.2.2).

2. **Validate that we are building the right thing.** Our teaching assistant (TA), Chris, pointed out that for this project, validation and usability matter more than verification. Correctly built lessons that the specialists disagree with, or summaries adults can't understand, are still a failure. We validate with the supervisor, the specialists we interviewed and adult usability sessions (see the Software Validation section).

3. **Show that Squigglr is usable and accessible.** Adults need to set up lessons and read summaries quickly, and the child's screens need to be easy to follow. We check this in usability sessions and against selected Web Content Accessibility Guidelines (WCAG) 2.2 success criteria.

4. **Make sure lessons and records behave correctly.** Stage changes, pass and fail decisions, attempt limits and adult control are the core of the product, and an adult who sees a wrong record or summary could make a bad decision for the child. We test these at the minimum, default and maximum of every value the adult can set, and against the formal specification in SRS Section 9.2.

5. **Protect children's data.** We keep records about children, so we check that a child is identified only by an identifier and an avatar, that no personal information is sent to the refinement service, and that only the right adults can see a child's records.

Some values are still to be determined (TBD) in the SRS, such as the performance limits and `REFINE_TOLERANCE`. For each one, this plan says how and when we will set it instead of leaving a placeholder (see the Symbolic Parameters appendix).

With five people and one academic year, we are leaving these out:

- **Testing with children.** No information is gathered from or about children and no study involves them (SRS CON-005 and CON-006), so every test and session uses adults or prepared attempts. This means the child's experience is judged by adults and the specialists rather than by children, which we accept as a limit of this plan.
- **Whether Squigglr improves children's drawing skills.** Showing that would take a long study with children, and Squigglr makes no such claim. We only check that the lessons work as specified and that the specialists think they make sense.
- **Third-party services and libraries.** We assume the authentication provider, the hosting and data storage service, the refinement model, the browsers and libraries like our testing frameworks are verified by the teams that make them. We only test how Squigglr uses them: what it sends, what it does with the result, and how it behaves when one is down. For example, we check that a refined drawing stays within `REFINE_TOLERANCE`, not how good the model's output looks.
- **Waiting Room features** (SRS Section 25), such as letter tracing, finger drawing and offline use, since they are not part of the product we are building. This includes full WCAG conformance; we only check the selected success criteria.
- **Every device and browser.** We test on the test iPad with a stylus, on the browsers in our end-to-end tests (Section 3.6), and on the other `SUPPORTED_DEVICES` combinations we have access to (T-PERF-007), not on every touchscreen device.

2.3 Custom Documents

[Summarize the CDs (Custom Documents) that were tackled by this project. CDs can include usability testing, code walkthroughs, user documentation,

formal proof, GenderMag personas, Design Thinking, etc. CDs should have already been approved by the course instructor as included in your problem statement. You can use a pull request to update your CDs (in TeamComposition.csv or Repos.csv) if your plan changes as a result of the VnV planning exercise. —SS]

2.4 Relevant Documentation

This plan builds on our other project documents. Here is how each one feeds into our VnV work:

- **Problem Statement and Goals** (Squigglr Team, 2026c). Defines the problem and the goals Squigglr has to meet. Validation checks Squigglr against this problem, not just against the SRS, so it is what we go back to when the supervisor or the specialists question whether a feature is needed.
- **Software Requirements Specification (SRS)** (Squigglr Team, 2026d). The main input to this plan. Every system test traces to an SRS requirement and uses its fit criterion as the pass condition (Section 4.3). The symbolic constants give the boundary values we test at, and the formal specification in SRS Section 9.2 gives the expected output of the lesson tests.
- **Hazard Analysis** (Squigglr Team, 2026b). Each hazard lists the requirements that mitigate it, which can be functional, usability or safety-critical (Hazard Analysis Section 6). The safety-critical ones get the safety tests and safety inspection (Section 4.2.2), and the rest are covered by their functional and usability tests. The causes of each hazard tell us what failures the tests should try to cause, like repeated failed attempts or a lost connection.
- **Development Plan** (Squigglr Team, 2026a). Names the language, coding standard, tools and continuous integration (CI) that our automated testing builds on (Section 3.6), and the lead roles behind some of our VnV roles. Its target-tablet experiment is where the first performance limits come from.
- **Module Guide (MG)** (Squigglr Team, 2027a). Due in January 2027. Splits Squigglr into modules. The unit tests are organized by these

modules, and the traceability between tests and modules checks that each one is covered. Design verification reviews it with the MG checklist.

- **Module Interface Specification (MIS)** (Squigglr Team, 2027b). Also due in January 2027. Gives each module's interface, which is what the unit tests call and check against.
- **VnV Report** (Squigglr Team, 2027c). Where we record the results of running this plan, including the measured values the performance tests report.

3 Plan

We start with who does what, then cover how we review the SRS, the design documents and this plan. After that come the plans for verifying the implementation through code walkthroughs, inspections and tests, and the tools that run those checks automatically (Section 3.6). The last part is validation: checking with the supervisor, the specialists and adult users that Squigglr is the right product.

3.1 Verification and Validation Team

Table 2 shows what each of us is responsible for verifying. Some roles come from our Development Plan leads: Huzaifah is Testing Lead, Shamil keeps CI running as DevOps Lead, and Ahmed plans the usability sessions as User Interface and User Experience (UI/UX) Lead. The rest come from the sections and tests each of us owns in this plan, and each of us still tests our own code. The teammate in the last column reviews each member's tests and pull requests, so nobody signs off on their own work.

Table 2: Verification and validation roles of the team members

Member and VnV role	**Verification responsibilities**	**Checked by**
Muhammad Huzaifah *Testing Lead and VnV coordinator*	Overall test strategy, plus the guided lesson and attempt record tests (`T-LRN`, `T-REC`). Leads code walkthroughs and the code inspections other than the safety inspection, records the safety inspection results, decides which tests run in CI, and keeps track of defects and coverage gaps.	Rayan
Rayan Nasrallah *Safety, performance and security lead*	Performance, safety-critical, operational, maintainability, security, cultural and compliance tests. Runs the safety inspection, which is the second check on every safety requirement (Section 4.2.2), and does not inspect code he wrote.	Huzaifah
Ahmed Elzaria *Design verification lead*	Creative drawing and progress summary tests (`T-CRT`, `T-PRG`), the unit test plan and the design document reviews. Plans the usability sessions with Shamil.	Moustafa
Shamil Canbolat *Requirements and validation lead*	Usability and accessibility tests (`T-UX`), SRS verification and software validation. Sets up the sessions with the supervisor and the specialists, sets up the automated testing tools, and keeps the CI build and test workflow running.	Ahmed
Moustafa Moustafa *VnV plan verification and quality assurance (QA) coordinator for this milestone*	Verifies this plan and does its final review against the VnV and Writing checklists. Turns review feedback into GitHub issues, takes notes at every review and validation session, and reviews the frontend code.	Shamil

People outside the team are also involved:

- **Dr. Christopher Anand (supervisor and client).** We give him a high-level update on the requirements, design and progress at our weekly meeting and record his feedback in the meeting issue. Shortly after Rev 0, we demo Squigglr to him so he can tell us whether it is what the project needs. His advice on children's safety and privacy goes straight into our requirements and tests, for example using an avatar instead of the child's name.

- **Domain specialists P1, P2 and P3.** Dr. Anand asked us (October 28 meeting) to go back to the specialists we interviewed for the SRS, or people with similar qualifications. They will try a working version of Squigglr and give feedback on the lesson guidance and feedback wording. They are volunteering their time, so we will keep the session short and work around their schedules.

- **Teaching assistant and peer teams.** They review the SRS, this plan and the design documents. We track each piece of their feedback as a GitHub issue with one owner.

- **Adult participants.** They take part in the usability sessions. We only test with adults, never children (SRS CON-005 and CON-006).

Our roles shift over the year. In the Fall, we write this plan and review the SRS and design documents. In the Winter, each of us runs the tests for our area as it gets built, and Shamil and Ahmed run the usability and specialist sessions. Like in the Development Plan, the QA coordinator rotates every milestone and is named in that milestone's tracking issue. If someone is unavailable or a role gets too heavy, we reassign the work at the next team meeting and update Table 2.

3.2 SRS Verification

[List any approaches you intend to use for SRS verification. This may include ad hoc feedback from reviewers, like your classmates (like your primary reviewer), or you may plan for something more rigorous/systematic. —SS]

[If you have a supervisor for the project, you shouldn't just say they will read over the SRS. You should explain your structured approach to the review. Will you have a meeting? What will you present? What questions

will you ask? Will you give them instructions for a task-based inspection? Will you use your issue tracker? —SS]

[Maybe create an SRS checklist? —SS]

3.3 Design Verification

[Plans for design verification —SS]

[The review will include reviews by your classmates —SS]

[Create a checklists? —SS]

3.4 Verification and Validation Plan Verification

Each review in Table 3 checks this plan in a different way, and nobody reviews a part they wrote.

Table 3: Reviews of this plan

Review	**Who**	**When**	**What it checks**
Pull request review	The teammate who checks the author's work (see the Verification and Validation Team section)	Every PR, before it merges	The changed sections, using the review questions below. A PR needs one approval from someone other than its author.
TA review	Chris, our TA	Informal meeting on October 28, 2026, and the marked submission	Our questions on how to write the plan, then whether the submitted plan is specific and feasible. His October 28 answers are listed below.

Review	Who	When	What it checks
Supervisor review	Dr. Anand	Supervisor meeting on October 28, 2026, where we discussed validation (#201), then our weekly meetings after that	A high-level summary of how we will validate Squigglr, to check we are validating what matters to him.
Peer review	The peer teams assigned by the course	After submission, due November 5, 2026	The whole plan, read by people outside the project. We ask them to use the VnV checklist.
Mutation check	Each member, on sections they did not write	Final QA (November 1–2, 2026) and before Rev 1	Whether our tests would actually catch realistic bugs (see below).
Final QA	Moustafa, except Moustafa's own sections, which Shamil checks	November 1–2, 2026	The whole plan against the VnV and Writing checklists and the rubric (see below).
Running the plan	The owner of each test	Rev 0 and Rev 1	Any test we cannot run exactly as written is a defect in this plan.

TA feedback from October 28. In an informal meeting, we asked Chris how to handle a few parts of the plan. His answers became rules for the whole plan:

- A value can stay TBD, but we have to say how it will be decided. Every TBD constant in the Symbolic Parameters appendix has to say how and when it is set, and Final QA checks this.
- There is no minimum number of tests per safety requirement, but we

have to justify the number we pick. The safety tests explain why one test plus the safety inspection is enough (Section 4.2.2).

- One parameterized test with a table of cases counts as one test, as long as it is easy to follow. Each case gets a label and its own expected output, so it can be cited on its own.
- Full branch coverage is a good goal for safety-critical modules, and matters less for the user interface (Section 3.6).
- For this project, validation and usability matter more than verification.

Review questions. Every reviewer, in a PR review or in Final QA, asks these about each test they check:

1. Could someone else run it without asking the author? The input is specific and the expected output is explicit.
2. Does it trace to a requirement, and is it in the traceability tables (Section 4.3)?
3. Does it use symbolic constants instead of magic numbers, and does each constant have a value or a method in the Symbolic Parameters appendix?
4. Does it try to break Squigglr, with boundary values, invalid input and failures, instead of only showing the normal case?
5. For a review or inspection: does it say who does it, when, and with which checklist?
6. Can we actually do it with our team, devices and schedule?

Mutation check. Mutation testing makes a small change to a program and checks that some test fails. We apply the same idea to the plan. Each member takes five requirements whose tests they did not write, including at least two safety requirements. For each one, they write down a small, realistic bug, such as a lesson ending one attempt after `ATTEMPT_LIMIT` instead of at it, or a refined point landing just outside `REFINE_TOLERANCE`. Then they find the test that would fail because of it. If no test would fail, or the test's input

does not include that case, they open an issue for the test's owner. The check passes when every bug is caught by at least one test. Each member needs about an hour for this.

Final QA. As QA coordinator for this milestone, Moustafa:

1. checks that every SRS requirement ID appears in the traceability tables, with a script that compares the IDs in the SRS with the ones in this plan;
2. checks that every TBD constant has a method, that no template comments are left, and that the PDF builds with no errors or undefined references;
3. goes through the VnV and Writing checklists and the rubric, one item at a time;
4. sends fixes in someone else's section to them as PR comments.

Tracking findings. Small fixes the team finds, like wording, go in PR comments. Everything else, and all feedback from the TA and peer teams, becomes a GitHub issue with the `review` label, one assignee, and what closes it. An issue closes when the fix is merged or when we record why we are not changing the plan.

3.5 Implementation Verification

The implementation is checked in two ways: by running it and by reading it. Running it covers:

- the system tests in this plan: the functional tests (T-LRN, T-CRT, T-REC and T-PRG) and the nonfunctional tests (T-UX, T-PERF, T-SAF, T-OPS, T-MAINT, T-SEC, T-CULT and T-COMP), traced to the requirements in Section 4.3;
- the unit tests for each module, chosen as described in Section 5 once the MIS is done (the design documentation is due on 18 January 2027);
- the automated checks in Section 3.6, which run the automatic parts of these tests on every PR, unless a test names a narrower trigger.

Reading it uses the techniques in Table 4. Dynamic tests only show behaviour on the inputs they choose. Reading finds paths that no test reaches, and checks what no test can, such as naming, hard-coded constants and the mapping to the MG and MIS. The safety inspection (SI, Section 4.2.2) covers the code tied to the safety-critical requirements and is not repeated here.

Table 4: Static verification of the implementation

Technique	**Who**	**When**	**What is read**	**Checklist**
PR code review	One teammate other than the author, normally the one who checks the author's work in Table 2. Database schema and access-policy changes are reviewed by the Backend Lead (Rayan Nasrallah) together with a teammate (Development Plan).	Every PR, before merge.	The changed code and the PR's unit tests.	Items marked PR in Table 5. The reviewer reads the unit tests against the rules in Section 5.

Technique	Who	When	What is read	Checklist
Code walk-through	The author presents, and the rest of the team follows and asks questions. The Testing Lead (Muhammad Huzaifah) chooses the code and the scenario. The notetaker (Moustafa Moustafa) records findings in the meeting issue.	45 minutes of the Sunday 10:00 team review on 22 November 2026, 6 December 2026, 24 January 2027, 21 February 2027 and 7 March 2027.	At most two modules, the ones changed most since the last walk-through, followed along one scenario.	Items marked W.
Code inspection (INS-LRN, INS-EVAL, INS-REC, INS-CRT)	The Testing Lead moderates. Two inspectors who did not write the code read it. The author answers questions.	Completed by 20 November 2026 (PH-1), 29 January 2027 (PH-2) and 12 March 2027 (PH-3).	The lesson rules; stroke capture and evaluation; record saving; the refinement result check and drawing history.	Items marked I, plus A1 to A11.
Safety inspection (SI)	Rayan runs it and Huzaifah records.	Before Rev 0 and Rev 1.	The code tied to the SAF requirements.	The SI steps in Section 4.2.2.

Table 5 shows how reviewers apply each item of the team's Code Checklist to Squigglr. The "Used in" column names the techniques that check the item: PR review (PR), walkthrough (W) and inspection (I).

Table 5: Code Checklist items and how reviewers apply them

#	Item	Used in	What the reviewer checks in Squigglr
1	Identifier names are consistent	PR, W, I	Names match the SRS and MIS. The stage names are demonstration, full outline tracing, endpoint-only guidance and independent drawing, as in SRS FR-LRN-002. Constants are named exactly as in SRS Section 4.2 (for example `STAGE_PASS`). Record fields are named as in SRS Section 7.2.
2	Identifier names are meaningful	PR, W, I	A short name such as `nf` is allowed only with a comment citing the SRS Section 9.2 symbol.
3	Source code in the src folder	PR	Application code is under `src/`. Each unit test sits next to its module as `<module>.test.ts`. End-to-end tests and fixtures are kept in `tests/` (fixtures in `tests/fixtures/`).
4	Comments on "what" not "how"	PR, I	Each exported function of the lesson rules, evaluation, record, refinement check and drawing history code states what it guarantees and cites its SRS requirement (for example "FR-LRN-021").
5	Avoid hard-coded constants	PR, I	No literal value of an SRS constant, such as 80, 8 or 5. Values come from the learner's settings, or from one constants module that uses the SRS names.
6	Consistent indentation	PR	The Prettier check passes; this item is not checked by hand.
7	Explicit identification of coding standard	PR, once	The ESLint and Prettier configuration files are committed, and the README links the Development Plan coding standard.

#	Item	Used in	What the reviewer checks in Squigglr
8	Code standard is followed	PR, I	ESLint and `tsc` pass. The reviewer checks what the tools cannot: `camelCase` and `PascalCase`, and a justification next to every `any` or lint suppression.
9	Parameters in the same order for all functions	PR, I	Learner, lesson, stage, attempt, in that order, everywhere.
10	Descriptive file names	PR	Each file is named after what it does.
11	Mapping to the module guide is clear	W, I, from the design documentation (18 January 2027)	Each inspected file belongs to one MG module, and the mapping is kept in the repository.
12	Mapping between MIS symbols and code symbols	W, I, from the design documentation (18 January 2027)	Each MIS access program of the inspected module has a function of the same name, with the same inputs and outputs.
13	Understandable to someone new to the project	W	Someone who did not write the code can explain the scenario's path after the walkthrough.
14	Esoteric language features avoided	PR, W	A simpler construct is used wherever one would do.
15	Incomplete code flagged	PR, I	Incomplete code is marked `TODO(#issue)` with an open issue, and no `TODO` remains in inspected code at a milestone.

Items added for the inspections. The Code Checklist asks for more items, so the inspections add these:

A1 (INS-LRN) Each row of the SRS Section 9.2 transition table maps to one identifiable code path. After a finished attempt, the rules are checked in the SRS order: pass, end at limit, extension, suggestion. In three places SRS Section 9.2 disagrees with other parts of the SRS: a go-back suggestion pending during practice, the counts when a stage is entered again, and when an extended lesson ends. The inspector records which reading the code follows and raises it as an SRS feedback issue, not as a major finding. The guided lesson tests avoid these cases.

A2 (INS-LRN) Apart from pass-based advancement and the step from the demonstration to practice, only an adult action that passed the adult check changes the lesson or the stage (I1, I2 and SRS CON-007).

A3 (INS-LRN, INS-EVAL) An interrupted (not finalized) or invalid attempt cannot reach the pass branch or change a count (SRS FR-LRN-009 and FR-LRN-010). An incomplete attempt cannot pass, whatever its result, but counts as a failed attempt (I3, FR-LRN-011 and FR-LRN-013).

A4 (INS-LRN) On every path, the evaluable attempts at a stage never exceed `ATTEMPT_LIMIT_MAX`, the extension attempts used never exceed `ATTEMPT_EXTENSION`, and the extension is granted at most once (I5).

A5 (INS-EVAL) Evaluation and saving use the strokes as captured, with no smoothing or refinement, and the lesson code imports nothing from the refinement code (I4, SRS BR-004 and BR-005).

A6 (INS-EVAL) Feedback and completion text come only from the lesson catalogue data, never from code literals or a model (FR-LRN-014, FR-LRN-022 and NFR-MAINT-001).

A7 (INS-REC) An attempt record copies the adult-set values in use instead of pointing at the current settings (FR-REC-004), and no code path updates a saved attempt record or an ended lesson record (FR-REC-009).

A8 (INS-REC) A retried save cannot create a second record, and each attempt is saved under the lesson record it was made in (FR-REC-008).

A9 (INS-CRT) A refinement result is checked as a whole before any of it is applied. It is applied only if it changes nothing but the paths of existing strokes and keeps every refined stroke within `REFINE_TOLERANCE` of its original; a failed, rejected or out-of-date result leaves the drawing unchanged (SRS FR-CRT-011, FR-CRT-012, FR-CRT-020 and FR-CRT-021; H-MUH-002). A stroke that an earlier refinement changed is not refined again unless the child edited it since (FR-CRT-014).

A10 (INS-CRT) Every edit, refinement and restore is one undoable change in the drawing history. The original strokes of each refinement are kept apart and never overwritten, also when the drawing is saved and reopened, so a restore returns them exactly (FR-CRT-007, FR-CRT-017, FR-CRT-018 and FR-CRT-024; H-MUH-001).

A11 (all) Every branch of the inspected files is reached by a test in this plan or a unit test, or the gap is listed from the coverage report (Section 3.6).

What each inspection covers.

- INS-LRN: the lesson rules (FR-LRN-009, FR-LRN-010, FR-LRN-015, FR-LRN-016, FR-LRN-018 to FR-LRN-021 and FR-LRN-023; SRS Section 9.2).
- INS-EVAL: stroke capture, finishing an attempt, classification, the pass decision and the choice of feedback (FR-LRN-007 to FR-LRN-014, FR-LRN-022 and FR-REC-002).
- INS-REC: creating and saving attempt and lesson records (FR-REC-001 and FR-REC-003 to FR-REC-010).
- INS-CRT: checking refinement results, and the creative drawing history with undo, redo and restore (FR-CRT-007, FR-CRT-008, FR-CRT-011, FR-CRT-012, FR-CRT-014, FR-CRT-017 to FR-CRT-021 and FR-CRT-024).

PH-1 inspects INS-EVAL and INS-LRN for the PH-1 code. INS-REC starts in PH-2, when FR-REC-004 to FR-REC-010 are built; in PH-1, FR-REC-001 and FR-REC-003 rely on PR review and on T-REC-001 and T-REC-003. INS-CRT also starts in PH-2, when FR-CRT-014, FR-CRT-019, FR-CRT-021 and FR-CRT-024 complete refinement and restore; in PH-1, the creative drawing code relies on PR review and on the T-CRT tests. PH-2 inspects all four in full, with items 11 and 12, because the MG and MIS are done on 18 January 2027. PH-3 inspects only the code changed since the commit of the PH-2 inspection.

Inspection procedure.

1. *Planning* (moderator, two working days before the meeting): open an issue labelled `review`. The issue names the files and commit, the SRS requirements, the two inspectors and the checklist items. The inspectors are chosen from people who did not write the files, according to the commit history. For INS-REC, one of them is the Backend Lead unless he wrote the code. If the Testing Lead wrote any of the files, the milestone QA coordinator moderates instead.

2. *Preparation* (each inspector alone, at most one hour): check the code against every item, and list each finding in the issue with its file, line and item.

3. *Meeting* (at most one hour; moderator, inspectors and author): go through the findings and mark each one major or minor. A finding is major if it could change a pass decision, a count, a stage, a saved record or what the child or adult sees, or if it breaks one of A1 to A10. A disagreement inside the SRS named in A1 is not major; it becomes an SRS feedback issue. Every other finding is minor. The author answers questions but does not argue findings.

4. *Rework* (author): each major finding becomes its own issue labelled `bug`, linked from the inspection issue and fixed by a PR. Each minor finding is fixed, or closed with a reason.

5. *Follow-up* (moderator): check every major fix. If there were three or more major findings, inspect the changed code again before the milestone.

The inspection passes when no major finding is open on its date. When a file is in both an inspection and the safety inspection at the same milestone, the team holds one meeting: the SI steps first, then A1 to A11.

Walkthrough procedure. The author shares the code on screen and follows one scenario from input to saved result, for example a failing attempt from the stylus events, through evaluation and the lesson rules, to the saved attempt record. Listeners say where they cannot follow (item 13), and findings become GitHub issues. The walkthroughs on 6 December 2026 and 21 February 2027 prepare for the course code-review interviews on 9 December 2026 and 23 February 2027 (Development Plan). Findings from those interviews are recorded the same way. From 24 January 2027, walkthroughs also check items 11 and 12.

Feasibility and reporting. An inspection takes about seven person-hours: two hours of preparation for the two inspectors, a one-hour meeting for four people, and one hour of planning and follow-up. The ten inspections (two in PH-1, four in PH-2 and four smaller ones in PH-3) take about 60 person-hours, or about twelve for each member over the year. Walkthroughs use time in the existing weekly meeting. The VnV Report lists each inspection and walkthrough with its date, commit, participants, findings by checklist item and severity, and status.

3.6 Automated Testing and Verification Tools

This section names the tools that check the code automatically. They follow the expected technology and coding standard in the Development Plan. They are planned, not yet installed; the DevOps Lead (Shamil Canbolat) sets them up when the application scaffold exists, and the Testing Lead (Muhammad Huzaifah) chooses which tests run where. Table 6 lists each tool, what it checks and when it runs.

Table 6: Automated testing and verification tools

Purpose	**Tool**	**How and when it is used**
Language	TypeScript, strict mode	Next.js front end and server code. `tsc --noEmit` runs locally and on every PR; any type error fails the check.
Coding standard and linter	ESLint with the Next.js Core Web Vitals and TypeScript ESLint configurations	The coding standard in the Development Plan. Runs on every PR; any error fails the check. Used as evidence for NFR-MAINT-004.
Formatter	Prettier (check mode)	Runs on every PR; any unformatted file fails the check (NFR-MAINT-004).
Unit and component tests	Vitest with React Testing Library	Unit tests for logic such as stroke evaluation and save handling, and component tests for screens. Runs locally before review and on every PR; any failing test fails the check.
Code coverage	`@vitest/coverage-v8`	Produces a statement and branch coverage report on every PR (see below).
End-to-end tests	Playwright, on Chromium, WebKit and Firefox	Complete user flows, including the automated tests in the nonfunctional system tests (for example network loss and repeated failed attempts). Runs on every PR once a flow exists. Emulation does not replace physical stylus testing, so stylus tests are also run by hand on the test iPad.
Continuous integration	GitHub Actions	Runs all checks above on PRs to `development` and `main` and after each merge. A PR cannot be merged while a required check fails. The existing `latex-pages.yml` workflow continues to build the documents.

Purpose	Tool	How and when it is used
Performance measurement	Safari Web Inspector (Timelines), Chrome DevTools (Performance panel) and the browser Performance API (`performance.now()`)	Measures stroke latency, refinement time and page load for the performance tests. Safari Web Inspector, run on a Mac, reads timings from the test iPad; Chrome DevTools is used for runs on a laptop. Timings logged by Playwright scripts make the measurements repeatable.
Network inspection	Chrome DevTools (Network panel)	Records requests for the HTTPS and third-party data tests (NFR-OPS-001, SAF-SHA-002).

Code coverage. The coverage report is attached to every PR as a GitHub Actions artifact, and the change in coverage is noted in the PR review. The team does not set a fixed coverage target, because high coverage alone does not show that tests are good (Development Plan). Instead, reviewers check that new or changed logic has tests, and that modules tied to a safety-critical requirement (the safety-critical tests below), such as attempt saving, stroke evaluation and access checks, have every branch covered. Full branch coverage of these modules is a goal, not a guarantee: some branches (for example, a server failure in the middle of a request) may be hard to reach in a test. The actual coverage is reported honestly in the VnV Report as a summary table of statement and branch coverage per module, and any safety-critical branch left uncovered is listed with the reason and the test or inspection that covers it instead.

Limits of the tools. Passing these checks shows that the code builds, follows the coding standard and passes its tests. It does not show that the tests are complete, so the code inspections planned under Implementation Verification remain necessary.

3.7 Software Validation

[If there is any external data that can be used for validation, you should point to it here. If there are no plans for validation, you should state that

here. —SS]

[You might want to use review sessions with the stakeholder to check that the requirements document captures the right requirements. Maybe task based inspection? —SS]

[For those capstone teams with an external supervisor, the Rev 0 demo should be used as an opportunity to validate the requirements. You should plan on demonstrating your project to your supervisor shortly after the scheduled Rev 0 demo. The feedback from your supervisor will be very useful for improving your project. —SS]

[For teams without an external supervisor, user testing can serve the same purpose as a Rev 0 demo for the supervisor. —SS]

[This section might reference back to the SRS verification section. —SS]

4 System Tests

These tests check Squigglr as a whole against the requirements in the SRS. Section 4.1 covers the functional requirements and Section 4.2 the nonfunctional ones, and Section 4.3 traces every test back to its requirement. Tests of individual modules come later, once the design is done (Section 5).

Every test has an ID `T-<AREA>-###`, where `AREA` is the requirement group it tests, and the same fields: Purpose, Type, Initial State, Input, Output, Test Case Derivation and How test will be performed. Values the SRS leaves open are written as symbolic constants, and the Symbolic Parameters appendix gives each value or says how it will be set.

When one rule needs several boundary cases, like a setting at its minimum, default and maximum, we write one parameterized test instead of a separate test for each case. Its input is a table with one row per case. Each row has a letter and its own expected output, and the derivation says why each output is right, so a single case can be cited as, for example, `T-LRN-005(c)`. Our TA, Chris, agreed that this counts as one specific test as long as it is easy to follow.

4.1 Tests for Functional Requirements

These tests cover the functional requirements in SRS Section 9, grouped the same way the SRS groups them: guided lessons (FR-LRN, SRS Sections 9.1

and 9.2), creative drawing (FR-CRT, Section 9.3), attempt records (FR-REC, Section 9.4) and progress summaries (FR-PRG, Section 9.5). Each test starts from prepared input, like a stroke sequence, a drawing or an attempt history, so its expected output is known before it runs. The lesson tests use the formal specification in SRS Section 9.2 to work out the expected state after each step.

4.1.1 Area of Testing1

[It would be nice to have a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. If a section covers tests for input constraints, you should reference the data constraints table in the SRS. —SS]

Title for Test

1. test-id1

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs. Output is not how you are going to return the results of the test. The output is the expected result. —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

4.1.2 Area of Testing2

...

4.2 Tests for Nonfunctional Requirements

These tests cover the nonfunctional requirements in SRS Sections 10 to 17, with one subsection per group: usability and accessibility, performance, safety-critical, operational and environmental, maintainability and support, security, cultural, and compliance. Most have a pass condition. Speed tests have one too, but one run can be slow by chance, so they repeat each run and report the results as tables and plots in the VnV Report. Where a requirement can't be tested by running Squigglr, like the wording in a template, we check it with a review or an inspection and say who does it, when, and with which checklist. The usability tasks and survey questions are in the Usability Survey Questions appendix.

4.2.1 Performance

These tests cover the performance requirements in SRS Section 12 (NFR-PERF-001 to NFR-PERF-005, NFR-PERF-008 and NFR-PERF-009). Timing thresholds are written as symbolic constants. The SRS leaves their values to this plan, so each constant in the Symbolic Parameters appendix states how its value is set and when.

Test environment. Squigglr is a web app, so in principle it runs on any device with a modern browser, and on any network. Testing every device and every connection is not realistic for a team of five, and it is not needed to show that the requirements are met. The team will instead run the timing tests on one tablet, an iPad with a stylus, which is the device the app is designed around (SRS ASM-003). The tests run in a controlled setting: the iPad is charged, Low Power Mode is off, every other app and browser tab is closed, automatic updates are paused, and the iPad uses the same Wi-Fi network for every run. Keeping these conditions the same means that a

change in a measured time comes from a change in Squigglr, not from the device or the network. Automatic tests that do not need a physical stylus run in Playwright on a team laptop under the same rules.

Why each test is repeated. Even in a controlled setting, one measurement can be slow by chance, for example when the browser collects garbage. Each timing test is therefore repeated `PERF_RUNS` times, and the VnV Report gives the minimum, median, 95th percentile and maximum of the measured times next to the threshold. A test passes only when the 95th percentile and the maximum are both within the threshold. This way a single slow run is still visible, but one unlucky run does not decide the result.

Limits. Because the tests use one device and one network, they show that Squigglr meets its timing requirements on the device it is designed for. They do not show how it performs on older devices or slower networks. Other browsers and devices are checked for correct operation, not speed, in T-PERF-007.

1. T-PERF-001: Stroke feedback latency

   Purpose: a child should see each stroke appear as they draw it; a visible delay makes drawing feel broken and can frustrate a child who is still building fine motor control.

   Type: Dynamic, automatic (Playwright) and manual (physical stylus).

   Initial State: a guided lesson is open at each stage where the child draws (tracing, reduced guidance and independent drawing) with no earlier attempts; no other tabs are open.

   Input: the prepared stroke files `line.json`, `circle.json` and `square-4-strokes.json`, replayed as pointer events by Playwright, `PERF_RUNS` times each; then `PERF_RUNS` strokes drawn by hand with the stylus on the iPad.

   Output: for every stroke, the time from the last pointer event to the first frame that shows the stroke is at most `FEEDBACK_LATENCY_MAX`.

   Test Case Derivation: NFR-PERF-001 requires visual acknowledgement within `FEEDBACK_LATENCY_MAX` of the final input point. Multi-stroke shapes are included because they create the most rendering work

per attempt. The replayed strokes give exactly the same input on every run, so they are the main measurement. A person cannot draw the same stroke twice, so the hand-drawn strokes are not meant to be identical: only the time to show each stroke is measured, not how accurate it is. They are there because replayed events skip the real stylus input path on the iPad, which could add its own delay.

How test will be performed: the replay script stamps `performance.now()` at the last pointer event and in the next `requestAnimationFrame` callback after the stroke is drawn. For the manual runs, the iPad is connected to a Mac and the same interval is read from a Safari Web Inspector timeline recording. Times are summarized as described above.

2. T-PERF-002: Refinement latency and canvas responsiveness

   Purpose: refinement runs on an outside service that can be slow, so this checks that the refined drawing arrives in time and that the child can keep drawing while it waits.

   Type: Dynamic, automatic.

   Initial State: the creative space is open with refinement on and the refinement service reachable.

   Input: the prepared drawings `house.json` and `sun.json` are replayed and finished, `PERF_RUNS` times each; during the wait, a further stroke is replayed every 200 ms.

   Output: the refined result appears within `REFINE_LATENCY_MAX` of finishing, and every stroke replayed during the wait is drawn and saved without an error.

   Test Case Derivation: NFR-PERF-002 requires both the time limit and an unblocked canvas while the refinement is pending.

   How test will be performed: the script records the time from the finish action to the first frame showing the refined result, and checks the canvas contents after each extra stroke.

3. T-PERF-003: Progress summary load time

Purpose: a parent or educator with a long history of saved attempts should not be left waiting for the progress summary to load.

Type: Dynamic, automatic.

Initial State: an adult account with one child profile whose history holds `HISTORY_SIZE_LARGE` attempts across three lessons, logged in, with the browser cache cleared.

Input: the adult opens the progress summary ten consecutive times.

Output: each time, the summary is fully visible within `SUMMARY_LOAD_MAX`.

Test Case Derivation: the fit criterion of NFR-PERF-003 names ten consecutive requests. A large history is used because load time grows with the number of records.

How test will be performed: Playwright measures from the click to the moment the last chart element is visible. The test is repeated with histories of 10 attempts and `HISTORY_SIZE_LARGE` attempts, and the two sets of times are reported together to show how load time grows with history size.

4. T-PERF-004: Deterministic stage result

Purpose: the adult relies on pass and fail results to choose the next level, so the same drawing must always get the same result.

Type: Dynamic, automatic (unit test in Vitest).

Initial State: the stage evaluation module is loaded with the criteria for each prepared stage.

Input: the prepared stroke files from T-PERF-001, plus `circle-near-miss.json`, a drawing just outside `STAGE_PASS`, each evaluated ten consecutive times.

Output: all ten results for each input are identical, including the pass or fail decision.

Test Case Derivation: NFR-PERF-004 requires the same strokes and criteria to give the same result. The near miss checks that a borderline result does not change between runs.

How test will be performed: a Vitest test calls the evaluation function directly and compares every result with the first. A second run in Playwright replays the same strokes through the user interface, to confirm that input handling does not change the result.

5. T-PERF-005: Submitting attempts after reconnection

   Purpose: home and school Wi-Fi can drop; a child's work done while offline must still reach the server once the connection returns, without being lost or saved twice.

   Type: Dynamic, automatic.

   Initial State: a guided lesson is open; the network is online.

   Input: the network is set offline in Playwright; the prepared attempt `circle.json` is completed; the network is restored. This is done three consecutive times, then repeated with three attempts completed while offline.

   Output: after each reconnection, every attempt completed offline is saved on the server exactly once, in the order made, and none is lost or duplicated.

   Test Case Derivation: NFR-PERF-005 names three consecutive runs. The three-attempt case checks the order of a queue and catches duplicates.

   How test will be performed: after each run, the script reads the child's attempt records through the test database and compares them with the attempts made. This test is shared with SAF-RAY-003 and SAF-SHA-003 (safety-critical tests below).

6. T-PERF-006: Concurrent session capacity

   Purpose: several classrooms or households may use Squigglr at once; this checks that the speed limits still hold under that load.

   Type: Dynamic, automatic (load test).

   Initial State: the preview deployment, with `CONCURRENT_CAPACITY` test adult accounts and one child profile each.

Input: a load script runs `CONCURRENT_CAPACITY` simulated sessions at once; each replays a lesson of ten attempts and then opens the progress summary.

Output: no request fails, and the measurements of T-PERF-001 to T-PERF-003, taken in one extra session during the load, stay within their thresholds.

Test Case Derivation: NFR-PERF-008 requires the speed limits to hold under the agreed load.

How test will be performed: the load script runs simulated sessions in Playwright; one further session takes the timing measurements. The run is repeated at a quarter, half and all of `CONCURRENT_CAPACITY`, and the times are plotted against the number of sessions in the VnV Report.

7. T-PERF-007: Squigglr keeps working until the end of the capstone

   Purpose: Squigglr must keep working on the supported devices and browsers at least until the end of the capstone, the final demo on 15 March 2027 (PH-3 in the SRS). Browsers update often, and an update can break a feature without any change to our code, so this checks every feature at each milestone.

   Type: Dynamic, manual.

   Initial State: the release build at each milestone: the proof-of-concept demo (PH-1, 23 November 2026), Rev 0 (PH-2, 1 February 2027) and the final demo (PH-3, 15 March 2027).

   Input: on the iPad and on each other combination in `SUPPORTED_DEVICES` that the team has access to, a team member follows the checklist of every feature: create an account and a child profile, run one guided lesson to the end, draw in the creative space, restore an original, and open the progress summary.

   Output: every checklist item works on every combination. Any combination that fails is removed from `SUPPORTED_DEVICES`, and OI-09 is updated.

   Test Case Derivation: the fit criterion of NFR-PERF-009.

How test will be performed: a team member runs the checklist by hand at each milestone. Running it at the two earlier milestones, not only at the final demo, means a browser update that breaks a feature is found while there is still time to fix it. The results are kept as a table of device by feature in the VnV Report.

4.2.2 Safety-Critical Requirements

These tests cover the 18 safety-critical requirements in SRS Section 12.2, which come from the Hazard Analysis. A failure here could harm a child's well-being, privacy or records, so each requirement gets two different kinds of evidence: a test below, and the safety inspection described next. Every test below is dynamic except T-SAF-004, which checks wording in templates and so is an inspection; T-SAF-016 combines a code review with recordings on the iPad. The tests try to break each safeguard, for example by having a child tap adult-only controls, cutting the connection during an attempt, or switching between children on a shared device. All tests use test accounts and prepared data only; no child takes part (SRS CON-006). In these tests, "the adult" means the parent or educator responsible for the child, and "the adult check" means the check in `ADULT_CHECK` (for example a PIN or password) that only the adult can complete.

Safety inspection (SI). Before each revision (Rev 0 and Rev 1), the code that implements each requirement below is inspected by two team members who did not write it. The Backend Lead (Rayan Nasrallah) runs the inspection and the Testing Lead (Muhammad Huzaifah) records the results. For each requirement, the inspectors:

1. find every code path that triggers the safeguard, using the test-to-requirement traceability table;
2. check that no path skips it, for example an adult-only action reachable from a URL or a save that reports success before the server responds;
3. check that a dynamic test exercises each path, and add one if not;
4. check that errors are handled (every failed request leaves the child's record unchanged and the adult is told) and that every input from the browser is validated on the server;

5. record each finding as a GitHub issue labelled `safety`, and re-inspect after the fix.

The inspection passes when every requirement has no open `safety` issue. Results are reported in the VnV Report.

Why this is enough. Testing cannot prove that a safeguard always works; it can only raise confidence that it does. One test per requirement, each with cases chosen to break the safeguard, plus an inspection that looks for paths the test missed, gives two independent checks. A second dynamic test is added wherever the inspection finds an untested path. The inspection covers only the code tied to these 18 requirements, because a full review of every module by two people is not realistic in the time the team has. The rest of the code is covered by the code checklist and the normal PR review.

How a lesson works. Several of these tests check the limits in a guided lesson, so this paragraph summarizes the lesson flow from SRS BR-002, UC-002 to UC-005 and the state machine in SRS Section 9.2. The adult picks the child and the lesson and completes the adult check. The child then watches a demonstration and works through three stages in order: tracing a full outline, drawing between endpoints, and drawing with no guide. An attempt that meets `STAGE_PASS` moves the child to the next stage automatically, and a pass at the last stage completes the lesson. Failed attempts are counted *per stage*, and the count starts again whenever the stage changes:

- after `DIFFICULTY_STREAK` failed attempts in a row, Squigglr suggests to the adult that the child go back a stage; the stage changes only if the adult completes the adult check and accepts, and the child keeps practising while the suggestion waits;
- after `ATTEMPT_LIMIT` failed attempts at one stage, the lesson ends automatically, unless the child is improving, in which case the child gets up to `ATTEMPT_EXTENSION` more attempts, never more than `ATTEMPT_LIMIT_MAX` in total at that stage;
- after `LESSON_IDLE_PROMPT` with no input, Squigglr shows the child a check-in, and after `LESSON_IDLE_END` with no input, the lesson ends; time without input never counts as a failed attempt.

Because ATTEMPT_LIMIT is always larger than DIFFICULTY_STREAK (SAF-RAY-008), the adult always receives the suggestion before the lesson ends: the suggestion is the early warning and the limit is the backstop. The adult sets these values for each child within ranges set by the team. The examples below use illustrative values (DIFFICULTY_STREAK = 3, ATTEMPT_LIMIT = 6, ATTEMPT_EXTENSION = 2, ATTEMPT_LIMIT_MAX = 8) to show the expected behaviour; the tests themselves use the ends of each range.

1. T-SAF-001: Adult-only actions need the adult check

   Purpose: choosing a lesson, choosing a starting stage, switching child and accepting a go-back suggestion belong to the adult, so a child must not be able to complete any of them.

   Type: Dynamic, manual and automatic. Covers SAF-RAY-001.

   Initial State: an adult account with two child profiles; child A is in a guided lesson, and a go-back suggestion is waiting for the adult.

   Input: on the lesson screen, the tester (acting as the child) taps every visible control, taps the screen 50 times at random positions, opens the web address of each adult-only action directly, uses the browser's back and forward buttons and reloads the page. Each time the adult check appears, the tester leaves it or enters a wrong answer. Finally, the tester (acting as the adult) completes the adult check and accepts the suggestion.

   Output: every adult-only action asks for the adult check, and none completes until the check is passed. The suggestion stays waiting and the child can keep drawing at the current stage. After the adult passes the check and accepts, the child moves back one stage.

   Example: child A fails three attempts in a row at the endpoints stage, and a go-back suggestion appears. Child A taps "accept"; Squigglr asks for the adult check, which child A cannot complete, so the child stays at the endpoints stage and keeps practising. The adult then completes the check and accepts, and child A moves back to the tracing stage.

   Test Case Derivation: SAF-RAY-001 and its hazards H-RAY-001 and H-RAY-006. Random taps imitate a child who cannot read exploring the screen.

How test will be performed: a Playwright script performs the taps, web addresses and navigation and checks the lesson state after each one; a tester repeats the taps by hand with the stylus on the iPad.

2. T-SAF-002: Notice on an unsupported device, browser or input

   Purpose: drawing results may be unreliable on a device or browser that is not in `SUPPORTED_DEVICES`, or when the child draws with a finger instead of a stylus, so the adult must be told and can decide whether to continue.

   Type: Dynamic, automatic and manual. Covers SAF-RAY-002.

   Initial State: the adult has chosen a lesson; it has not started.

   Input: (a) the lesson is started once with each combination in `SUPPORTED_DEVICES`, and once with each of three combinations that are not in it: an older browser version, a phone-sized screen, and a browser without pointer event support; (b) on the iPad, a supported lesson is started and the first stroke is drawn with a finger; (c) the same, with the first stroke drawn with the stylus.

   Output: (a) for each unsupported combination, a notice the adult can read appears before the first attempt; for each supported combination, no notice appears; (b) the notice appears after the first stroke; (c) no notice appears.

   Test Case Derivation: the fit criterion of SAF-RAY-002 (H-RAY-002). A browser cannot tell whether a stylus is connected before it is used, so case (a) checks only what the browser reports (version, screen size and pointer event support), and cases (b) and (c) check the input type the browser reports for the first stroke (pen or touch).

   How test will be performed: for (a), Playwright sets the browser version, screen size and pointer event support for each combination and checks for the notice; for (b) and (c), a tester draws on the iPad, once with a finger and once with the stylus.

3. T-SAF-003: Attempts kept until the server confirms them

   Purpose: an attempt must never look saved when it is not, or the adult would make decisions from incomplete records.

Type: Dynamic, automatic. Covers SAF-RAY-003 and SAF-SHA-003.

Initial State: a guided lesson is open and online.

Input: (a) the connection is cut during an attempt; (b) the connection is cut after the attempt is finished but before the server confirms the save, and the page is then reloaded, and separately closed and reopened, while offline; (c) the server returns an error for `FAILED_SAVES` saves in a row; (d) `FAILED_SAVES` saves succeed normally.

Output: in (a) to (c), no attempt is shown as saved before the server confirms it, the adult is told which attempts are not saved, and every attempt is saved once the connection or server is back; in (d), no failure message appears.

Test Case Derivation: SAF-RAY-003 and SAF-SHA-003 (H-RAY-002, H-SHA-003). Case (d) checks that the warning does not appear when nothing failed.

How test will be performed: Playwright turns the network off and makes the server return errors, then compares the server's records with the attempts made.

4. T-SAF-004: No diagnosis or comparison in summaries and exports

   Purpose: Squigglr is a practice tool, not an assessment, so no summary or export may label, diagnose or compare a child, and each must say so.

   Type: Static (inspection), manual. Covers SAF-RAY-004.

   Initial State: the templates for every summary and export at that revision.

   Input: one team member who did not write the templates reads each one.

   Output: every template states that it describes practice only and is not an assessment or diagnosis, and none uses a diagnostic label, judging wording (for example "struggling", "behind" or "below average") or a comparison with other children.

   Test Case Derivation: the fit criterion of SAF-RAY-004 (H-RAY-003), which asks for an inspection of every summary and export template.

The team does not intend to add diagnoses, so the realistic risks are a new template that leaves out the required statement and wording that judges the child; a read of each template catches both.

How test will be performed: once per revision (Rev 0 and Rev 1), as part of the safety inspection; any problem is recorded as a GitHub issue labelled `safety`.

5. T-SAF-005: Lesson ends automatically at the attempt limit

   Purpose: a child must not be left repeating a stage they keep failing, so the lesson must end by itself at the limit the adult set.

   Type: Dynamic, automatic. Covers SAF-RAY-005.

   Initial State: a child profile with `ATTEMPT_LIMIT` and `ATTEMPT_LIMIT_MAX` at the lowest values in their ranges; the test is then repeated with the highest values.

   Input: (a) at the tracing stage, the failing attempt `circle-fail.json` is replayed until the lesson ends; (b) the child passes tracing after a few failures, and the failing attempt is replayed at the endpoints stage until the lesson ends; (c) a sequence of attempts that improves enough to earn an extension is replayed until the lesson ends.

   Output: the lesson ends after exactly `ATTEMPT_LIMIT` failed attempts at one stage, or `ATTEMPT_LIMIT_MAX` when extended, with no action from the child; failures at an earlier stage do not count at the new stage; and the saved record matches the attempts made, including the furthest stage reached.

   Example: the sixth failed tracing attempt ends the lesson. In case (b), a child who passed tracing after four failures still gets six attempts at the endpoints stage. In case (c), an improving child gets attempts 7 and 8, and the lesson ends after the eighth failure.

   Test Case Derivation: SAF-RAY-005 (H-RAY-004). Testing both ends of the ranges follows the supervisor's advice to test at the minimum and maximum (SRC-008).

   How test will be performed: Playwright counts the attempts and checks that no attempt is accepted after the lesson ends.

6. T-SAF-006: Limit endings are shown to the adult across repeated lessons

   Purpose: each time a lesson is done, it is a new lesson record, so the adult must see how often the lesson ended at the limit across all of them; doing the lesson again must not hide earlier failures.

   Type: Dynamic, automatic. Covers SAF-RAY-006.

   Initial State: as in T-SAF-005.

   Input: the adult starts the lesson and it is run until it ends at the limit; the adult then starts the same lesson again, and this is repeated so the lesson is done three times. In the third run, an improving sequence earns an extension before the lesson ends at the limit.

   Output: the adult's view of that lesson shows three lesson records, each ending at the limit, one extension, and every attempt from all three records.

   Example: the adult sees "ended at the limit: 3 times; extended: once" and all $6 + 6 + 8 = 20$ attempts.

   Test Case Derivation: SAF-RAY-006 (H-RAY-005).

   How test will be performed: the script compares the counts and the list of attempts in the adult's view with the runs made.

7. T-SAF-007: Attempts saved only to the active child

   Purpose: on a shared device, one child's attempts must never be saved to another child's record.

   Type: Dynamic, automatic and manual. Covers SAF-RAY-007.

   Initial State: one adult account with three child profiles (A, B and C) on one device.

   Input: the adult switches between the children in the order A, B, A, C, and each child makes two attempts; the connection is cut once during child B's turn.

   Output: every practice screen shows the active child's avatar, and every attempt and suggestion is saved under the child whose avatar was shown when it was made.

Test Case Derivation: SAF-RAY-007 (H-RAY-006). The lost connection checks that a delayed save still goes to the right child.

How test will be performed: Playwright checks each child's records; a tester checks by hand that each avatar can be recognized without reading.

8. T-SAF-008: Adult-set values stay within their ranges

   Purpose: a mistyped or extreme value could give a child an impossible lesson, so each value must stay within the range the team set and only the adult may change it.

   Type: Dynamic, automatic. Covers SAF-RAY-008.

   Initial State: a new child profile with no values chosen.

   Input: (a) no value is set; (b) each value is set to its minimum, its maximum, one below the minimum and one above the maximum; (c) `ATTEMPT_LIMIT` is set equal to `DIFFICULTY_STREAK`, and `ATTEMPT_LIMIT_MAX` is set one below `ATTEMPT_LIMIT`; (d) a change is tried without completing the adult check.

   Output: (a) each value is the middle of its range; (b) in-range values are saved and out-of-range values are refused; (c) both settings are refused; (d) nothing changes. The values in use are shown with the child's records.

   Example: if the team-set range for `ATTEMPT_LIMIT` were 4 to 10, an unset value would be 7, values 4 and 10 would be saved, and 3 and 11 would be refused. With `DIFFICULTY_STREAK` = 3, an `ATTEMPT_LIMIT` of 3 is refused, because the lesson would end before the adult received the go-back suggestion.

   Test Case Derivation: the fit criterion of SAF-RAY-008 (H-RAY-007), using boundary values on each side of each range.

   How test will be performed: a Vitest test checks the validation function with every case, and Playwright repeats cases (b) to (d) through the settings screen and through direct server requests.

9. T-SAF-009: Duplicate input events counted once

Purpose: a stylus event reported twice must not change a drawing's result, or a child could pass or fail for no real reason.

Type: Dynamic, automatic. Covers SAF-AHM-001.

Initial State: a guided lesson at the tracing stage.

Input: `square-4-strokes.json` is replayed as recorded, then with every input event sent twice; then `overlap-2-strokes.json`, two separate strokes that cross, is replayed.

Output: the recorded points, classification and pass result are the same with and without the duplicates, and both crossing strokes are kept.

Test Case Derivation: the fit criterion of SAF-AHM-001 (H-AHM-002).

How test will be performed: Playwright replays both versions and compares the saved records.

10. T-SAF-010: Drawing kept when the canvas changes size

    Purpose: rotating the iPad or resizing the window must not distort a drawing or change its result.

    Type: Dynamic, automatic. Covers SAF-AHM-002.

    Initial State: a guided lesson with an attempt partly drawn.

    Input: the window is resized and the iPad is rotated during the attempt; then input events with positions outside the canvas, missing positions (`NaN`) and very large values are sent.

    Output: the order and relative positions of the points are unchanged, and the classification is the same as without resizing; none of the invalid points is counted or makes the attempt pass.

    Test Case Derivation: the fit criterion of SAF-AHM-002 (H-AHM-003).

    How test will be performed: Playwright resizes the drawing area and compares the scaled points with the original; a Vitest test sends the invalid values to the input handler.

11. T-SAF-011: Every data request checks the account

    Purpose: one family's or school's data must never be reachable from another account.

Type: Dynamic, automatic. Covers SAF-SHA-001, with T-SEC-001.

Initial State: two adult accounts, A and B, each with a child profile and saved attempts.

Input: logged in as A, the tester sends requests to read, change and delete B's child profiles, attempts, settings and summaries to every server endpoint, then repeats them with no session and with an expired session.

Output: every request is refused, and none of B's data is returned or changed.

Test Case Derivation: the fit criterion of SAF-SHA-001 (H-SHA-001).

How test will be performed: a script lists every endpoint from the code and sends each request; the list is checked in the safety inspection, so a new endpoint cannot be missed.

12. T-SAF-012: No personal information sent to outside services

    Purpose: children's and adults' personal information must not leak to outside services, including the refinement service.

    Type: Dynamic, manual. Covers SAF-SHA-002.

    Initial State: an account whose adult name and email, and the child profile's identifier, are unique marker strings.

    Input: the tester uses every feature, including refinement, while recording all network traffic.

    Output: no request to an outside service contains any marker string, and requests go only to approved destinations.

    Test Case Derivation: the fit criterion of SAF-SHA-002 (H-SHA-002). Unique markers make any leak easy to find.

    How test will be performed: the test runs in Chrome on a laptop; the DevTools Network panel is exported as a HAR file and searched for the markers and for destinations that are not approved.

13. T-SAF-013: Logout after inactivity

Purpose: an adult who walks away from a shared device must not leave their account and their children's data open.

Type: Dynamic, automatic. Covers SAF-SHA-006.

Initial State: an adult is logged in.

Input: (a) no activity for `IDLE_TIMEOUT`; (b) activity just before `IDLE_TIMEOUT`, then no activity.

Output: the adult is logged out after `IDLE_TIMEOUT` of inactivity and not before, and any later request using that session is refused.

Test Case Derivation: SAF-SHA-006 (H-SHA-001). Case (b) checks that activity restarts the timer.

How test will be performed: Playwright uses a controlled clock to move time forward and checks the session.

14. T-SAF-014: Feedback does not draw the missing part

    Purpose: feedback should show the child what is missing without drawing it for them, so the record shows only the child's own work.

    Type: Dynamic, automatic. Covers SAF-MOU-001.

    Initial State: a guided lesson at each predefined difficulty.

    Input: `square-missing-side.json`, an attempt with one side missing.

    Output: the feedback shows where the side is missing, and the strokes on screen and in the record are the same before and after the feedback.

    Test Case Derivation: the fit criterion of SAF-MOU-001 (H-MOU-004).

    How test will be performed: Playwright compares the stroke data before and after the feedback is shown.

15. T-SAF-015: Assisted, independent and creative work kept apart

    Purpose: the adult must be able to tell what the child drew with an on-screen guide from what they drew alone, so progress is not overstated.

    Type: Dynamic and static, manual. Covers SAF-MOU-002.

    Initial State: a prepared history of one lesson record holding 10 assisted attempts, 10 independent attempts, and 5 creative space drawings made separately.

Input: every summary and record view is opened.

Output: assisted and independent attempts are counted and shown separately, no creative space drawing appears, and no view ranks or rates the child by the number of attempts.

Example: a lesson record with 10 successful attempts while tracing and 2 while drawing alone is summarized as "10 with a guide, 2 without", never as "12 successful drawings".

Test Case Derivation: the fit criterion of SAF-MOU-002 (H-MOU-005). The assisted or independent label belongs to each attempt, not to the lesson: following FR-LRN-012, an attempt is assisted when it is drawn with an on-screen guide (the tracing or endpoints stage) and independent when it is drawn with no guide. One lesson record normally holds both kinds. Help from an adult outside Squigglr, such as guiding the child's hand, is not detected or recorded, and this test does not cover it.

How test will be performed: a tester checks each view against the prepared history, and a Vitest test of the summary function compares the counts.

16. T-SAF-016: No more than three flashes per second

Purpose: rapid flashing can trigger seizures, so no child-facing screen may flash more than three times in any one second.

Type: Static (code review) and dynamic, manual. Covers SAF-MOU-003.

Initial State: the code for every animation and transition on child-facing screens, and Squigglr open on the iPad with a guided lesson ready.

Input: (a) a team member who did not write it reviews every animation and transition used on child-facing screens, which are all defined in Squigglr's own code and style sheets; (b) on the iPad, the screens that animate (all feedback, the lesson-end message, the inactivity check-in and the screen transitions) are screen-recorded.

Output: (a) no animation or transition changes brightness or colour more than three times in any one second; (b) stepping through each

recording frame by frame, no one-second window has more than three flashes.

Test Case Derivation: the fit criterion of SAF-MOU-003 and WCAG 2.2 success criterion 2.3.1 (World Wide Web Consortium, 2023). Because every animation comes from our own code, reviewing that code finds a flashing effect before it reaches a screen; the recordings then confirm the result on the device, without recording every screen.

How test will be performed: once per revision (Rev 0 and Rev 1), as part of the safety inspection; any problem is recorded as a GitHub issue labelled `safety`.

17. T-SAF-017: Check-in and lesson ending after inactivity

    Purpose: a child may walk away during a lesson, and on a shared device another child could carry on under their profile, so an idle lesson must check in with the child and then end, with a clear record for the adult.

    Type: Dynamic, automatic and manual. Covers SAF-RAY-009.

    Initial State: a guided lesson at the endpoints stage, with `LESSON_IDLE_PROMPT` and `LESSON_IDLE_END` at the lowest values in their ranges; the test is then repeated with the highest values.

    Input: (a) no input for `LESSON_IDLE_PROMPT`, then a touch; (b) no input for `LESSON_IDLE_END`; (c) two failed attempts, then no input until the lesson ends.

    Output: (a) the check-in appears at `LESSON_IDLE_PROMPT`, the touch dismisses it, and the lesson continues at the same stage; (b) the check-in appears, then the lesson ends at `LESSON_IDLE_END`, the progress is saved, and the adult's view shows the lesson ended for inactivity; (c) the record shows exactly two failed attempts, so the idle time added none.

    Example: with illustrative values of 60 seconds for the check-in and 120 seconds for the end, a child who stops drawing sees an animated character at 60 seconds; if nobody touches the screen, the lesson ends at 120 seconds and the adult sees "ended: no activity".

    Test Case Derivation: the fit criterion of SAF-RAY-009 (H-RAY-006). Case (c) checks that inactivity is never counted as failure.

How test will be performed: Playwright uses a controlled clock to move time forward and checks the screen and the saved record; a tester checks by hand on the iPad that the check-in can be understood without reading and does not flash.

4.2.3 Operational and Environmental

These tests cover SRS Section 13 (NFR-OPS-001 to NFR-OPS-003): how Squigglr reaches users through the browser and the outside services it depends on.

1. T-OPS-001: HTTPS only

   Purpose: all data between the browser and the server must be encrypted, so children's records cannot be read in transit.

   Type: Dynamic, automatic and manual. Covers NFR-OPS-001.

   Initial State: the preview deployment.

   Input: in Chrome on a laptop, the tester uses every feature with the DevTools Network panel recording, then opens the `http://` address of each page. (DevTools cannot record traffic on the iPad, and the server's behaviour does not depend on the device.)

   Output: every request uses `https://`; every `http://` address redirects to HTTPS or is refused.

   Test Case Derivation: the fit criterion of NFR-OPS-001.

   How test will be performed: the HAR from the normal HTTPS feature run is searched for any `http://` request; the intentional `http://` probes are checked separately for a redirect or refusal, and a Playwright test checks the redirects on every release.

2. T-OPS-002: Error message when a provider is down

   Purpose: when an outside service is down, the adult should get a clear message instead of a broken screen.

   Type: Dynamic, automatic. Covers NFR-OPS-002.

   Initial State: an adult on the login page, and separately an adult in a lesson.

Input: requests to the authentication provider, and then to the hosting and storage service, are blocked to imitate an outage, and then unblocked.

Output: an error message is shown for as long as the provider is unavailable, no data is lost, and normal use resumes when it returns. Refinement outages are covered by the FR-CRT tests.

Test Case Derivation: the fit criterion of NFR-OPS-002.

How test will be performed: Playwright blocks the provider's domain with request routing and checks the page after each step.

3. T-OPS-003: No installation

   Purpose: families and schools should be able to use Squigglr in a browser without installing anything.

   Type: Dynamic, manual. Covers NFR-OPS-003.

   Initial State: a fresh browser profile, with no extensions, on the iPad and on each other combination in `SUPPORTED_DEVICES` that the team has access to.

   Input: the tester opens the address, creates an account and runs a lesson.

   Output: everything works with no download, plugin, extension or install prompt.

   Test Case Derivation: the fit criterion of NFR-OPS-003.

   How test will be performed: run together with T-PERF-007 and recorded in the same table.

4.2.4 Maintainability and Support

These checks cover SRS Section 14 (NFR-MAINT-001 to NFR-MAINT-004). Most are about the code and the process rather than running behaviour, so most are reviews or CI checks.

1. T-MAINT-001: Lessons are data, not code

Purpose: new lessons should be added without changing code, so the app can grow after the capstone.

Type: Static (code review) and dynamic, manual. Covers NFR-MAINT-001.

Initial State: the lesson catalogue at Rev 0.

Input: (a) a reviewer searches the source code for lesson content (shapes, stage order, feedback wording); (b) a test lesson for a new shape is added as a lesson file, edited and removed.

Output: (a) no lesson content is found in the source code; (b) the new lesson appears, changes and disappears in the app with no code change or rebuild, and its attempts are scored by the existing scoring.

Example: a lesson file holds the target outline as a list of points, the guides for each stage (full outline and endpoints) and the feedback messages. One scoring function compares the child's strokes with whatever outline it is given (how far the drawn points are from the outline and how much of it was covered), so a triangle lesson can be added as a new file in the same way as the circle and square lessons.

Test Case Derivation: NFR-MAINT-001. This covers lessons that use the existing scoring. A lesson that needs a new kind of scoring, for example letters, where stroke order matters, needs new code and is outside this test. How maintainers update lessons is still open in the SRS, so the add, edit and remove step will follow the SRS wording once it is settled.

How test will be performed: the review is done by a team member who did not write the catalogue module.

2. T-MAINT-002: Finding the module for a lesson behaviour

   Purpose: a new developer should be able to find the code behind a lesson behaviour quickly, so future fixes are practical.

   Type: Static (walkthrough), manual. Covers NFR-MAINT-002.

   Initial State: the code and the Module Guide (MG) at Rev 0.

   Input: a team member who did not write the lesson logic gets three questions, for example "where is the go-back suggestion triggered?",

and uses only the MG and the repository to find the module that answers each.

Output: each module is found within `LOCATE_TIME_MAX`.

Test Case Derivation: the fit criterion of NFR-MAINT-002.

How test will be performed: once, at Rev 0, taking about 15 minutes; any question that takes longer becomes an issue to improve the MG or the module names.

3. T-MAINT-003: Every module has a written interface

   Purpose: each module should state what it offers, what it takes in and what it gives back, so a team member can use another module without reading its code.

   Type: Static (document review), manual. Covers NFR-MAINT-003.

   Initial State: the GitHub issues that create a top-level module, and the Module Interface Specification (MIS).

   Input: the Testing Lead checks each of those issues.

   Output: every issue asks for an interface description (the functions the module offers, their inputs and outputs, and what must be true before and after each call), and each description matches the MIS.

   Example: the issue that creates the scoring module asks for a description such as "evaluate(strokes, target outline) returns the distance and coverage measures and whether the attempt meets STAGE_PASS", and the MIS describes the same function.

   Test Case Derivation: the fit criterion of NFR-MAINT-003, plus the match with the MIS that the requirement's title names.

   How test will be performed: done as part of the MIS review in the Design Verification plan.

4. T-MAINT-004: CI blocks style violations

   Purpose: code that breaks the coding standard should be stopped automatically before it is merged.

   Type: Dynamic, automatic. Covers NFR-MAINT-004.

Initial State: the CI pipeline described under Automated Testing and Verification Tools.

Input: two test PRs: one with an ESLint error (an unused variable) and one with a Prettier violation (wrong indentation).

Output: CI fails on both and blocks the merge; after the fix, it passes.

Test Case Derivation: the fit criterion of NFR-MAINT-004 names two different violations.

How test will be performed: the DevOps Lead opens both PRs once the pipeline is set up, and repeats this whenever the lint rules change.

4.2.5 Security

These tests cover SRS Section 15 (NFR-SEC-001 to NFR-SEC-006). They use test accounts only. A child profile holds only a unique identifier generated by Squigglr and an avatar chosen by the adult or the child (SRS BR-001, NFR-SEC-004).

1. T-SEC-001: An adult can only reach their own children

   Purpose: an adult must only ever reach their own children's profiles and records.

   Type: Dynamic, automatic. Covers NFR-SEC-001, with T-SAF-011.

   Initial State: accounts A and B, each with one child profile and attempts.

   Input: as A, read and modify B's child profile and attempt records through direct server requests, using B's record IDs.

   Output: every request is refused, and A's own data is unchanged.

   Test Case Derivation: the fit criterion of NFR-SEC-001.

   How test will be performed: a script sends the requests with A's session; the access policies in the database are also reviewed in the safety inspection.

2. T-SEC-002: Logout ends the session

Purpose: after logging out, nobody using the same device can continue the session.

Type: Dynamic, automatic. Covers NFR-SEC-002.

Initial State: an adult is logged in and the session token is copied.

Input: the adult logs out; the copied token is then used for an authenticated request, three times.

Output: all three requests are refused.

Test Case Derivation: the fit criterion of NFR-SEC-002.

How test will be performed: Playwright reads the token before logging out and sends the requests with it afterwards.

3. T-SEC-003: Audit event for a blocked change to a saved record

   Purpose: saved attempts are the record the adult relies on; any blocked attempt to change one must be logged.

   Type: Dynamic, automatic. Covers NFR-SEC-003.

   Initial State: a saved attempt record.

   Input: three requests to change the record, sent one after another.

   Output: every request is refused, each creates an audit event, and no event contains the record's contents.

   Example: a request to change a saved attempt's result from fail to pass is refused, and the audit event records the time, the account and the record's identifier, but not the drawing or the result.

   Test Case Derivation: the fit criterion of NFR-SEC-003.

   How test will be performed: the script reads the audit log through the team's database console and checks each event.

4. T-SEC-004: Minimal personal data

   Purpose: collecting less personal data lowers the harm if data is ever exposed.

   Type: Static (inspection), manual. Covers NFR-SEC-004.

   Initial State: the screen for adding a child and the database schema.

Input: a reviewer lists every field on the screen for adding a child and every column of the child profile in the schema.

Output: the only thing entered is the choice of avatar, and the only child profile columns are the identifier, the avatar and the adult-set values; there is no name, birth date, photo or other child-identifying field.

Example: a field asking for the child's first name, even an optional one, fails the test.

Test Case Derivation: the fit criterion of NFR-SEC-004. The schema is included because a field can be stored without being on the screen.

How test will be performed: the review is done with the PIPEDA review in T-COMP-001.

5. T-SEC-005: Login failures and timeouts are logged privately

   Purpose: failed logins and timeouts are logged so problems can be found, without exposing private details in the logs.

   Type: Dynamic, automatic. Covers NFR-SEC-005.

   Initial State: a test adult account.

   Input: ten failed logins and ten logouts by inactivity (using the controlled clock from T-SAF-013); then a request for the audit log as that adult.

   Output: all twenty events are in the server-side log with a time stamp, and the adult's request is refused.

   Test Case Derivation: the fit criterion of NFR-SEC-005.

   How test will be performed: Playwright triggers the events and the script reads the log through the team's database console.

6. T-SEC-006: Text inputs are sanitized

   Purpose: text typed into the app must not be able to run code or reach the database.

   Type: Dynamic, automatic. Covers NFR-SEC-006.

Initial State: every text field and every server endpoint that accepts text.

Input: the probe list `injection-probes.txt`, built from the OWASP SQL injection and XSS filter-evasion cheat sheets, for example `' OR '1'='1` and `<script>alert(1)</script>`.

Output: no probe changes a query or runs a script; each is stored or shown as plain text or refused.

Test Case Derivation: the fit criterion of NFR-SEC-006.

How test will be performed: Playwright enters each probe in every field and checks the page for script execution, and a script sends each probe to every text endpoint.

4.2.6 Cultural

These tests cover SRS Section 16 (NFR-CULT-002): cultural sensitivity of symbols and imagery.

1. T-CULT-001: Style review of symbols and imagery

   Purpose: symbols and images should be suitable for families from different cultures.

   Type: Static (review), manual. Covers NFR-CULT-002 (SRS Section 16).

   Initial State: every icon, avatar, image and colour used to mean success or failure, at Rev 0.

   Input: two team members, and the supervisor at the Rev 0 demonstration, review each item against a checklist: hand gestures, animals, colours, religious or national symbols, and food.

   Output: no item has a known negative meaning in communities likely to use Squigglr, or every concern raised is fixed and approved by the reviewers.

   Test Case Derivation: the fit criterion of NFR-CULT-002.

   How test will be performed: concerns are recorded as GitHub issues and the review is repeated for new images before Rev 1.

4.2.7 Compliance

These reviews cover SRS Section 17 (NFR-COMP-001, NFR-COMP-002 and NFR-COMP-005): the Canadian and Ontario privacy laws that apply to Squigglr, and the selected WCAG 2.2 success criteria.

1. T-COMP-001: PIPEDA review

   Purpose: Squigglr collects personal information in Canada, so its handling must meet PIPEDA.

   Type: Static (document and code review), manual. Covers NFR-COMP-001.

   Initial State: a data inventory listing every personal-information field, where it is stored and where it is sent.

   Input: the inventory, checked against each of the ten PIPEDA fair information principles (Government of Canada, 2000).

   Output: each principle is recorded as met or not applicable, with none unmet.

   Test Case Derivation: the fit criterion of NFR-COMP-001.

   How test will be performed: two team members do the review at Rev 0, so any problem can still be fixed, and repeat it at the final demo (PH-3, 15 March 2027), using the network findings from T-SAF-012 and the field list from T-SEC-004.

2. T-COMP-002: Ontario MFIPPA review

   Purpose: Ontario school boards may use Squigglr, so its handling of student data must meet MFIPPA, the privacy act that covers school boards.

   Type: Static (document review), manual. Covers NFR-COMP-002.

   Initial State: the same data inventory, with retention and disclosure added.

   Input: the inventory, checked against MFIPPA Part II, which covers the collection, use, retention and disclosure of personal information.

   Output: no unmet obligation is recorded.

Test Case Derivation: the fit criterion of NFR-COMP-002.

How test will be performed: done with T-COMP-001; any obligation the team cannot judge is raised with the supervisor before adoption by a school board.

3. T-COMP-003: Selected WCAG 2.2 criteria review

   Purpose: the WCAG 2.2 criteria the SRS selects make Squigglr usable by children and adults with different abilities, so each one must be met.

   Type: Static (review) and dynamic, manual. Covers NFR-COMP-005.

   Initial State: the release build at the final demo (PH-3), on the iPad.

   Input: two team members check every screen against each WCAG 2.2 success criterion selected through NFR-UX-001, NFR-UX-004, NFR-UX-005 and NFR-UX-018 to NFR-UX-023 (World Wide Web Consortium, 2023).

   Output: each selected criterion is recorded as met on every screen, with none unmet.

   Test Case Derivation: the fit criterion of NFR-COMP-005, which asks for a review team. Full WCAG conformance is a stretch goal in the SRS, so only the selected criteria are checked.

   How test will be performed: the review is first done at Rev 0, so problems can still be fixed, and repeated at PH-3; each unmet criterion is recorded as a GitHub issue.

4.3 Traceability Between Test Cases and Requirements

Every functional and nonfunctional requirement in the SRS has at least one test or named check, and the most important ones have two. The tables below map each requirement to its tests. A letter after a test ID, like `T-LRN-005 (c)`, points to one case of a parameterized test. Waiting Room features (SRS Section 25) are not listed, since they are out of scope.

Table 7 maps each performance, safety-critical, operational, maintainability, security, cultural and compliance requirement to the tests that cover it. “SI” is the safety inspection described with the safety-critical tests. Every

safety-critical requirement has at least two kinds of evidence, and several other requirements are also covered by a second test.

Table 7: Traceability for nonfunctional requirements

Requirement (SRS)	**Tests**
NFR-PERF-001	T-PERF-001, T-PERF-006
NFR-PERF-002	T-PERF-002, T-PERF-006
NFR-PERF-003	T-PERF-003, T-PERF-006
NFR-PERF-004	T-PERF-004 (unit and UI)
NFR-PERF-005	T-PERF-005, T-SAF-003
NFR-PERF-008	T-PERF-006
NFR-PERF-009	T-PERF-007, T-OPS-003
SAF-RAY-001	T-SAF-001, SI
SAF-RAY-002	T-SAF-002, SI
SAF-RAY-003	T-SAF-003, T-PERF-005, SI
SAF-RAY-004	T-SAF-004, SI
SAF-RAY-005	T-SAF-005, SI
SAF-RAY-006	T-SAF-006, SI
SAF-RAY-007	T-SAF-007, SI
SAF-RAY-008	T-SAF-008, SI
SAF-RAY-009	T-SAF-017, SI
SAF-AHM-001	T-SAF-009, SI
SAF-AHM-002	T-SAF-010, SI
SAF-SHA-001	T-SAF-011, T-SEC-001, SI
SAF-SHA-002	T-SAF-012, T-COMP-001, SI
SAF-SHA-003	T-SAF-003, SI
SAF-SHA-006	T-SAF-013, T-SEC-005, SI
SAF-MOU-001	T-SAF-014, SI
SAF-MOU-002	T-SAF-015, SI
SAF-MOU-003	T-SAF-016, SI
NFR-OPS-001	T-OPS-001
NFR-OPS-002	T-OPS-002
NFR-OPS-003	T-OPS-003, T-PERF-007
NFR-MAINT-001	T-MAINT-001
NFR-MAINT-002	T-MAINT-002
NFR-MAINT-003	T-MAINT-003
NFR-MAINT-004	T-MAINT-004
NFR-SEC-001	T-SEC-001, T-SAF-011
NFR-SEC-002	T-SEC-002

Requirement (SRS)	Tests
NFR-SEC-003	T-SEC-003
NFR-SEC-004	T-SEC-004, T-COMP-001
NFR-SEC-005	T-SEC-005
NFR-SEC-006	T-SEC-006
NFR-CULT-002	T-CULT-001
NFR-COMP-001	T-COMP-001
NFR-COMP-002	T-COMP-002
NFR-COMP-005	T-COMP-003

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That

can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

References

Government of Canada. Personal information protection and electronic documents act (PIPEDA), S.C. 2000, c. 5. https://laws-lois.justice.gc.ca/eng/acts/P-8.6/, 2000. Last amended 2015.

Squigglr Team. Development Plan for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/DevelopmentPlan/DevelopmentPlan.pdf, 2026a.

Squigglr Team. Hazard Analysis for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/HazardAnalysis/HazardAnalysis.pdf, 2026b.

Squigglr Team. Problem Statement and Goals for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/ProblemStatementAndGoals/ProblemStatement.pdf, 2026c.

Squigglr Team. Software Requirements Specification for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/SRS/SRS.pdf, 2026d.

Squigglr Team. Module Guide for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/Design/SoftArchitecture/MG.pdf, 2027a. In preparation, due January 2027.

Squigglr Team. Module Interface Specification for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/Design/SoftDetailedDes/MIS.pdf, 2027b. In preparation, due January 2027.

Squigglr Team. Verification and Validation Report for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/VnVReport/VnVReport.pdf, 2027c. Due March 8, 2027.

World Wide Web Consortium. Web content accessibility guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/, 2023. W3C Recommendation.

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

The constants below are used by the nonfunctional tests. The SRS leaves the timing thresholds to this plan; their values are set by measurement rather than chosen in advance, so each entry says how and when its value is set. Milestones use the SRS phase names: PH-1 is the proof-of-concept demo (23 November 2026), PH-2 is Rev 0 (1 February 2027) and PH-3 is the final demo (15 March 2027). Each constant includes its unit, so the tests name the constant alone (for example, "within `FEEDBACK_LATENCY_MAX`").

Table 8: Symbolic parameters for nonfunctional tests

Constant	Value or how it is set
`FEEDBACK_LATENCY_MAX`	Milliseconds. Set at the end of PH-1 from the target-tablet experiment in the Development Plan, run on the test iPad: the team measures the stroke latency of the first working build, checks whether drawing feels immediate, and sets the limit as the highest latency it accepts plus a margin, so that ordinary strokes of an accepted build do not fail. After that the limit is fixed and does not follow later builds. Recorded in the SRS constants table (NFR-PERF-001).
`REFINE_LATENCY_MAX`	Seconds. Set in PH-2 from the measured response time of the chosen refinement service (NFR-PERF-002).
`SUMMARY_LOAD_MAX`	Seconds. Set in PH-2 from the first measurement of T-PERF-003 (NFR-PERF-003).
`CONCURRENT_CAPACITY`	Sessions. Set in PH-2 from the number of schools and households expected during the capstone (NFR-PERF-008).

Constant	Value or how it is set
LOCATE_TIME_MAX	Minutes. Set by the team before the first T-MAINT-002 walkthrough (NFR-MAINT-002).
IDLE_TIMEOUT, FAILED_SAVES, SUPPORTED_DEVICES, ADULT_CHECK, STAGE_PASS, ATTEMPT_LIMIT, ATTEMPT_LIMIT_MAX, ATTEMPT_EXTENSION, DIFFICULTY_STREAK, LESSON_IDLE_PROMPT, LESSON_IDLE_END	As defined in the SRS constants table.
PERF_RUNS	Number of repeated runs per timing test on the test iPad. Proposed value: 30, enough to report a stable median and 95th percentile.
HISTORY_SIZE_LARGE	Number of attempts in the large prepared history. Proposed value: 500, well above a term of daily practice.

Prepared test data. The nonfunctional tests use stroke files that will be recorded once on the test iPad, before the first test run in PH-1, and stored in the repository under `tests/fixtures/`: `line.json`, `circle.json`, `square-4-strokes.json`, `circle-near-miss.json`, `circle-fail.json`, `square-missing-side.json`, `overlap-2-strokes.json`, `house.json` and `sun.json`, plus the probe list `injection-probes.txt`. Each file holds the pointer events (position, pressure, time) for one attempt.

6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

7.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

7.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

7.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

Individual Disclosures

Rayan: I used Anthropic's Claude Opus 5.5 at medium effort, accessed through Claude Code, while preparing my sections of this document (the automated testing and verification tools, the performance, safety-critical, operational, maintainability, security, cultural and compliance tests, the traceability table, the symbolic parameters and my reflection). I made the testing decisions in my sections myself. A major use of AI was researching testing tools and approaches: I used the model to find tools that fit our planned technology stack, compare the options for each kind of test, and work out which suited our project best, then chose the tools myself. I also used it to identify which privacy and compliance standards apply to the data Squigglr collects, such as PIPEDA and the Ontario act that covers school boards (MFIPPA), and to work out strategies for the more technical tests, such as checking that all traffic uses HTTPS, trying to read or change another account's data, and probing text inputs for injection; I checked each standard and decided which tests to include myself. I also used it to check my test cases against the SRS requirements, the Hazard Analysis and the notes from our supervisor and TA meetings, to turn my notes and decisions into draft wording, to improve sentence structure, clarity and flow, and to help with LaTeX formatting. Through Claude Code, it also helped manage the repository (branches, commits and merges). I reviewed every AI-assisted passage before including it.

8 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

   Team: Something that really went well was talking to Chris, our TA, and Dr. Anand, our supervisor, before the plan was finished. In our meeting with Chris on October 28, 2026, we asked about the things we were unsure of: whether thresholds we cannot set yet can be decided by measurement, how many tests a safety requirement needs, whether we need a fixed code coverage target, and whether one parameterized test can cover several boundary cases. His answers helped us narrow down which testing strategies to use and how to run certain tests, and showed us what in our first drafts was not clear yet, for example that every TBD value in the SRS should point to the method in this plan, and that we needed to justify why one test plus a safety inspection gives enough confidence. Chris also told us that validation and usability matter more than verification for this project, which is why they come right after safety in our objectives. Dr. Anand told us the same day to go back to the specialists we interviewed, or people with similar qualifications, to validate the app.

   Our process also went better than last time. In the SRS and Hazard Analysis we had many merge conflicts, mostly because everyone added

rows to the same revision history table. This time Rayan suggested that everyone send their revision history entries to Moustafa, who is doing the final quality check and adds them all at the end. We also started from one set of shared notes in all five VnV issues, so everyone knew which sections and test IDs they owned from the start. Rayan, Huzaifah and Moustafa split their parts into small sub-issues so each piece could be reviewed on its own, and Rayan's five branches merged together without conflicts when tested.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   Team: The main pain point was that writing the plan kept sending us back to the SRS. Every test has to match a requirement, so each time a meeting or a test changed how something should work, we had to go back through the SRS, change it, and then check that our tests still matched it. The main changes came from two meetings:

   - *Our supervisor meeting on October 28, 2026.* Dr. Anand advised that we should not collect children's personal details, so a child's profile now holds only a generated identifier and an avatar instead of a first name and username. This changed the business rules, the data dictionary, a use case, a security requirement and the Hazard Analysis, as well as the security test.
   - *Our TA meeting on the same day.* Chris advised that a value left as TBD should say how it will be decided, so every TBD performance value in the SRS now points to the measurement method in this plan, and the other TBD constants get a method in the Symbolic Parameters appendix.

   We resolved this by keeping all of these SRS changes in one issue and one pull request, separate from the VnV branches, and by re-reading each affected test against the updated requirement before marking a branch ready for review.

   Another pain point was that all five of us were writing the same `.tex` file at the same time. Sections on separate branches could not see each other until they merged, and some parts depended on others. For example, Moustafa's section introductions had to be stacked on top of

the front matter pull request. We resolved this by writing only in our own sections and rebasing onto `main` as other parts merged.

Our shared notes also told us to use the template's Control field, but Rayan's tests, the first ones merged, used Purpose and Type instead. Chris had told us we are not bound by the template's format, so we kept Purpose and Type and asked everyone to use the same fields.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

   Rayan: For my part of the plan (performance, safety-critical, security and compliance tests), I need three things I do not yet have:

   - *End-to-end test automation with Playwright*, in particular simulating network loss, replaying recorded stylus input and controlling the clock. Most of my safety tests depend on these, for example T-SAF-003, T-SAF-013 and T-SAF-017.
   - *Security testing of a web back end*: checking access policies in the database, reusing session tokens and sending injection probes. As Backend Lead, I am responsible for the code these tests target.
   - *Performance measurement on the iPad*: taking repeatable timings in the browser and summarizing them fairly, so that the thresholds the SRS leaves open can be set from evidence.

   Moustafa: I need to learn how to actually check a screen against WCAG. So far I have only used WCAG to write requirements, like NFR-UX-018 to NFR-UX-023. Writing "text needs a 4.5:1 contrast ratio" is easy, but T-COMP-003 has two of us check every screen against those criteria, and I review the frontend code, so I should be catching these problems before that review. This matters even more for us because we only evaluate with adults (SRS CON-005), so no child will be there to show us that a button is too small or that an icon makes no sense. The skill has two parts:

   - *Using accessibility tools*: a contrast checker for NFR-UX-019 and an automated checker for NFR-UX-020, and knowing what these

tools miss.

- *Checking by hand*: measuring touch targets for NFR-UX-018, using VoiceOver on the iPad for NFR-UX-020, and finishing the adult tasks with only a keyboard for NFR-UX-023.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

   Rayan:

   - *Playwright.* Two approaches: work through the official Playwright documentation and examples on network emulation and clocks, or learn from the first tests the Testing Lead writes. I will start with the documentation and write T-PERF-005 as a first test as soon as the scaffold exists. This is because the network and clock features are exactly what my tests need, and writing a real test early shows quickly what I am missing.
   - *Security testing.* Two approaches: follow the OWASP Web Security Testing Guide for the checks that apply to us, or rely on the hosting provider's security advice and default settings. I will use the OWASP guide, because it gives a checklist we can repeat each release, while default settings alone would not show us that our own access rules are correct.
   - *Performance measurement.* Two approaches: learn the browser developer tools (Safari Web Inspector for the iPad, Chrome DevTools on a laptop), or write timing code with the browser Performance API. I will do both, starting with the Performance API in Playwright scripts so the measurements can be repeated, and using Safari Web Inspector to understand any slow result on the iPad.

   Moustafa: Two approaches: work through the W3C's WCAG 2.2 pages for the criteria we selected and try each check with the tools, or learn by doing hands-on passes of our own screens, on the iPad with VoiceOver and on a laptop with only the keyboard. I will start with the W3C pages, because T-COMP-003 has to record each criterion as

met or not, so I need to know exactly what passing means. Full WCAG conformance is not required, so the list is short enough to get through before the first review at Rev 0. After that I will do hands-on passes on the first frontend screens I review, because a checker can tell me a control has a label, but not whether the label makes sense to someone who can't see the icon.